

GitHub to SQL Server CI/CD Pipeline

Complete Beginner Guide from Scratch

Introduction: What You Will Learn

This guide teaches you how to connect Visual Studio to SQL Server and automatically deploy SQL scripts using GitHub Actions.

We start from nothing. No assumptions.

Fundamental Concepts

- Visual Studio: Software where you write and edit code
- SQL Server: Database server where your data lives
- GitHub: Website where you store your code in the cloud
- GitHub Actions: Automation tool that runs tasks automatically
- Repository: Folder on GitHub that stores your files with version history
- Git: Software that tracks changes to your files

PART 1: Install Required Software

Step 1.1: Install Git

Git tracks changes to your files.

Go to: <https://git-scm.com/download/win>

Click: Download 64-bit Git for Windows

Run the installer

Click Next through all screens

Click Install

Click Finish

Verify Git is installed:

Open Command Prompt

Type: `git --version`

You should see: git version 2.x.x (or similar)

Step 1.2: Install Visual Studio

Visual Studio is where you write code and manage your project.

If you already have Visual Studio: Continue to next step

If you need to install:

Go to: <https://visualstudio.microsoft.com/downloads/>

Download: Visual Studio Community (free)

Run installer

Select: Desktop development with C# (includes Git tools)

Click Install

Step 1.3: Install SQL Server Management Studio (SSMS)

SSMS is where you manage your SQL Server.

Go to: <https://learn.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms>

Download latest SSMS

Run installer

Click Install

Step 1.4: Verify SQL Server is Installed and Running

Open SSMS

Server name: Leave as (local) or your server name

Authentication: Windows Authentication

Click Connect

If you connect successfully: Your SQL Server works

PART 2: Create a Local Folder for Your Project

What We Are Creating

We will create a folder that will become your GitHub repository.
This folder will store your SQL scripts and GitHub configuration files.

Step 2.1: Create Local Folder

On your computer, choose a location where you store projects.
Example: C:\Users\YourName\Documents\GitHub
Create a new folder named: SQL-CICD-DEMO
Full path will be: C:\Users\YourName\Documents\GitHub\SQL-CICD-DEMO
Note: Replace YourName with your actual Windows username

Step 2.2: Create Required Subfolder Structure

Inside SQL-CICD-DEMO folder, create these subfolders:

SQL-CICD-DEMO (main folder)
└─ Scripts (new folder - for SQL files)
└─ Database (new subfolder)
└─ (SQL files will go here)
└─ .github (new folder - GitHub configuration)
└─ workflows (new subfolder)
└─ (Workflow files will go here)
└─ (Other files like README.md)

How to create these folders in Windows:

1. Open File Explorer
2. Navigate to SQL-CICD-DEMO folder
3. Right-click > New > Folder
4. Type: Scripts
5. Repeat for .github folder
6. Inside Scripts, create Database subfolder
7. Inside .github, create workflows subfolder

PART 3: Open Project in Visual Studio

Step 3.1: Open Visual Studio

Launch Visual Studio

Step 3.2: Open Folder

Click: File > Open > Folder

Navigate to: SQL-CICD-DEMO folder (you just created)

Click: Select Folder

Visual Studio will open with your folder structure visible

Step 3.3: View Folder Structure in Visual Studio

On the left side, you will see "Solution Explorer"

It shows:

- SQL-CICD-DEMO (main folder)
 - Scripts
 - Database
 - .github
 - workflows

PART 4: Initialize Git Repository Locally

What is Git Initialization

Initializing Git tells Git to start tracking changes in this folder.
This creates hidden .git folder that stores version history.

Step 4.1: Open Terminal in Visual Studio

In Visual Studio, click: Terminal > New Terminal
You will see a terminal window at the bottom

Step 4.2: Initialize Git

In terminal, type:

```
git init
```

Press Enter

Output: Initialized empty Git repository in C:\...\SQL-CICD-DEMO\.git

This means Git is now tracking this folder

Step 4.3: Configure Git (One Time Setup)

Type these commands (one by one):

```
git config user.name "Your Name"
```

(Replace "Your Name" with your actual name)

```
git config user.email your.email@example.com (Replace with your actual email)
```

These tell Git who is making changes

PART 5: Create Your First SQL Script

Step 5.1: Create SQL File

In Visual Studio, right-click Scripts > Database folder

Select: New File

Name it: 001_CreateTables.sql

Visual Studio will open the file for editing

Step 5.2: Write Sample SQL Script

Paste this SQL code into your file:

```
CREATE TABLE dbo.Users  
(  
    UserID INT PRIMARY KEY IDENTITY(1,1),
```

```
UserName NVARCHAR(100) NOT NULL,  
Email NVARCHAR(255) NOT NULL,  
CreateDate DATETIME DEFAULT GETDATE()  
);  
GO
```

```
INSERT INTO dbo.Users (UserName, Email)  
VALUES ('TestUser', 'test@example.com');  
GO
```

```
SELECT * FROM dbo.Users;  
GO
```

Save file: Press Ctrl+S

Step 5.3: Test SQL Script Manually

Before deploying via GitHub, test it manually in SQL Server
Open SQL Server Management Studio (SSMS)
Connect to your local SQL Server
Create new database: TestDB (if not exists)
Open New Query window
Copy the SQL code from your file
Paste into query window
Click Execute (or press F5)
If successful: Table created and data inserted

PART 6: Create GitHub Workflow File

What is a Workflow

A workflow is a file that tells GitHub Actions what to do automatically.
It runs when you push code to GitHub.
It executes your SQL scripts on your SQL Server.

Step 6.1: Create Workflow File

In Visual Studio, right-click .github > workflows folder
Select: New File
Name it: deploy-sql.yml

Step 6.2: Paste Workflow Configuration

Copy this into the file:

```
name: Deploy SQL Scripts
on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: self-hosted
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Execute SQL Script
        shell: powershell
        run: |
          sqlcmd -S "localhost" -d "TestDB" -U "SQLUser" -P "YourPassword" -i
          ".\Scripts\Database\001_CreateTables.sql"
```

Save file: Press Ctrl+S

Note: You will need to update the credentials in this file later

PART 7: Add Files to Git

Step 7.1: Stage Files for Commit

In Visual Studio terminal, type:
git add .

Press Enter

This stages all your files (marks them for commit)

Step 7.2: Create First Commit

Type:

```
git commit -m "Initial commit with SQL scripts and GitHub workflow"
```

Press Enter

Output shows files added

Step 7.3: Check Git Status

Type:

```
git status
```

Press Enter

Output: On branch main, nothing to commit, working tree clean

This means all changes are committed

PART 8: Create GitHub Account and Repository

Step 8.1: Create GitHub Account

Go to: <https://github.com>

Click: Sign up

Enter email address

Create password

Enter username (this is your GitHub identity)

Verify email

You now have a GitHub account

Step 8.2: Create New Repository on GitHub

Log into GitHub

Click: + icon (top right)

Select: New repository

Fill in:

Repository name: SQL-CICD-DEMO

Description: CI/CD pipeline for SQL Server deployment

Visibility: Private (only you can see it)

Click: Create repository

GitHub creates your remote repository in the cloud

Step 8.3: Get Repository URL

After creating repository, GitHub shows you the URL

Look for: HTTPS URL

Example: <https://github.com/YourUsername/SQL-CICD-DEMO.git>

Copy this URL - you will need it next

PART 9: Connect Local Repository to GitHub

Step 9.1: Add Remote Repository

In Visual Studio terminal, type:

```
git remote add origin https://github.com/YourUsername/SQL-CICD-DEMO.git
```

(Replace YourUsername with your actual GitHub username)

Press Enter

This tells Git where your remote repository is

Step 9.2: Rename Default Branch

Type:

```
git branch -M main
```

Press Enter

This ensures your branch is named "main" (GitHub default)

Step 9.3: Push to GitHub

Type:

```
git push -u origin main
```

Press Enter

GitHub will ask for authentication

GitHub creates a personal access token (PAT)

If successful:

Output shows files pushed to GitHub

Step 9.4: Verify on GitHub

Go to: <https://github.com/YourUsername/SQL-CICD-DEMO>

You should see:

- Scripts folder with your SQL file
- .github folder with workflow file
- Your files are now in the cloud

PART 10: Add GitHub Secrets

What are GitHub Secrets

Secrets store sensitive information (passwords) securely on GitHub. Your workflow file can use these secrets without exposing them.

Step 10.1: Navigate to Secrets Settings

Go to: <https://github.com/YourUsername/SQL-CICD-DEMO>

Click: Settings tab (top menu)

Left sidebar: Secrets and variables

Click: Actions

Click: New repository secret

Step 10.2: Add SQL Server Secrets

Create these four secrets (one by one):

1. Secret 1: SQL Server Hostname

Name: SQL_SERVER_HOST

Value: localhost (or your SQL Server name)

2. Secret 2: Database Name

Name: SQL_DATABASE

Value: TestDB

3. Secret 3: SQL Username

Name: SQL_USERNAME

Value: Your SQL login username

4. Secret 4: SQL Password

Name: SQL_PASSWORD

Value: Your SQL login password

PART 11: Install GitHub Self-Hosted Runner

What is a Self-Hosted Runner

A runner is a computer that executes GitHub Actions.
A self-hosted runner is your personal Windows machine.
It has access to your local SQL Server.

Step 11.1: Download Runner

Go to: <https://github.com/YourUsername/SQL-CICD-DEMO/settings/actions/runners>
Click: New self-hosted runner
Select: Windows
Select: x64
Click: Download
Save to: C:\GitHub-Runner

Step 11.2: Extract Files

Create folder: C:\GitHub-Runner
Right-click zip file
Select: Extract All
Extract to: C:\GitHub-Runner

Step 11.3: Configure Runner

Open PowerShell as Administrator
Navigate: `cd C:\GitHub-Runner`
Type: `.\config.cmd`
Press Enter

Follow the prompts:
When asked for runner name: Type `sql-server-runner`
When asked for labels: Press Enter (skip)
When asked for work directory: Press Enter (accept default)

Step 11.4: Run the Runner

In same PowerShell window, type:
`.\run.cmd`
Press Enter

Wait for message: Listening for Jobs
Keep this PowerShell window open

Your runner is now active and ready to execute workflows

PART 12: Update Workflow with Credentials

Step 12.1: Update Workflow File

In Visual Studio, open: `.github\workflows\deploy-sql.yml`

Replace the `sqlcmd` line with:

```
sqlcmd -S "${{ secrets.SQL_SERVER_HOST }}" -d "${{ secrets.SQL_DATABASE }}" -U "${{ secrets.SQL_USERNAME }}" -P "${{ secrets.SQL_PASSWORD }}" -i ".\Scripts\Database\001_CreateTables.sql"
```

This uses GitHub Secrets instead of hardcoded credentials

Step 12.2: Commit and Push Changes

In Visual Studio terminal, type:

```
git add .  
git commit -m "Update workflow with GitHub Secrets"  
git push origin main
```

PART 13: First Automated Deployment Test

Step 13.1: Make a Small Change

In Visual Studio, edit: Scripts\Database\001_CreateTables.sql

Add a comment at the top:

```
-- This is a test deployment from GitHub Actions
```

Save file: Ctrl+S

Step 13.2: Commit and Push

In terminal, type:

```
git add .
```

```
git commit -m "Test automated deployment"
```

```
git push origin main
```

Step 13.3: Watch Workflow Execute

Go to: <https://github.com/YourUsername/SQL-CICD-DEMO>

Click: Actions tab

Look for your latest commit

Watch the workflow run

Green checkmark = Successful deployment

Red X = Failed (click to see error)

Step 13.4: Verify in SQL Server

Open SQL Server Management Studio

Check if table was created or updated

Success: Your CI/CD pipeline is working!

Summary: Your Complete Pipeline

You now have a fully automated pipeline:

1. You edit SQL scripts in Visual Studio
2. You commit changes with git
3. You push to GitHub
4. GitHub detects push and triggers workflow
5. GitHub sends workflow job to your runner
6. Runner executes SQL script on your SQL Server
7. Database is updated automatically
8. No manual deployments needed

Key Concepts to Remember

- Visual Studio = Code editor
- Git = Tracks file changes locally
- GitHub = Cloud repository (backup + sharing)
- GitHub Actions = Automation trigger
- Self-Hosted Runner = Your computer executing workflows
- SQL Server = Your database where scripts execute